DM Models 1 — Homework 2 Report

Task 1 — Titanic preprocessing, model tuning, and comparison

Preprocessing used. I used the standard Titanic training set (891 rows) and built a preprocessing pipeline that: (1) extracted **Title** from passenger names, (2) created **FamilySize**, **IsAlone**, and **CabinKnown**, (3) treated **Pclass**, **Sex**, **Embarked**, and **Title** as categorical variables, (4) imputed missing numeric values with the median and missing categorical values with the most frequent value, and (5) applied one-hot encoding to categorical features so the tree-based models could use them directly.

Tuned decision tree. Best hyperparameters: criterion = gini, max_depth = 5, min_samples_split = 2, min_samples_leaf = 1, ccp_alpha = 0.005.

Tuned random forest. Best hyperparameters: n_estimators = 200, max_depth = 8, min_samples_split = 2, min_samples_leaf = 1, max_features = sqrt.

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean	Std.
Decision Tree	0.832	0.820	0.787	0.831	0.837	0.822	0.018
Random Forest	0.844	0.843	0.826	0.848	0.837	0.840	0.008

Conclusion for Task 1. Random Forest performed better than Decision Tree on 5-fold cross-validation: **0.840 vs. 0.822**. It was also more stable across folds (smaller standard deviation: 0.008 vs. 0.018). The single decision tree is easier to interpret, but the random forest generalizes better because averaging many decorrelated trees reduces variance.

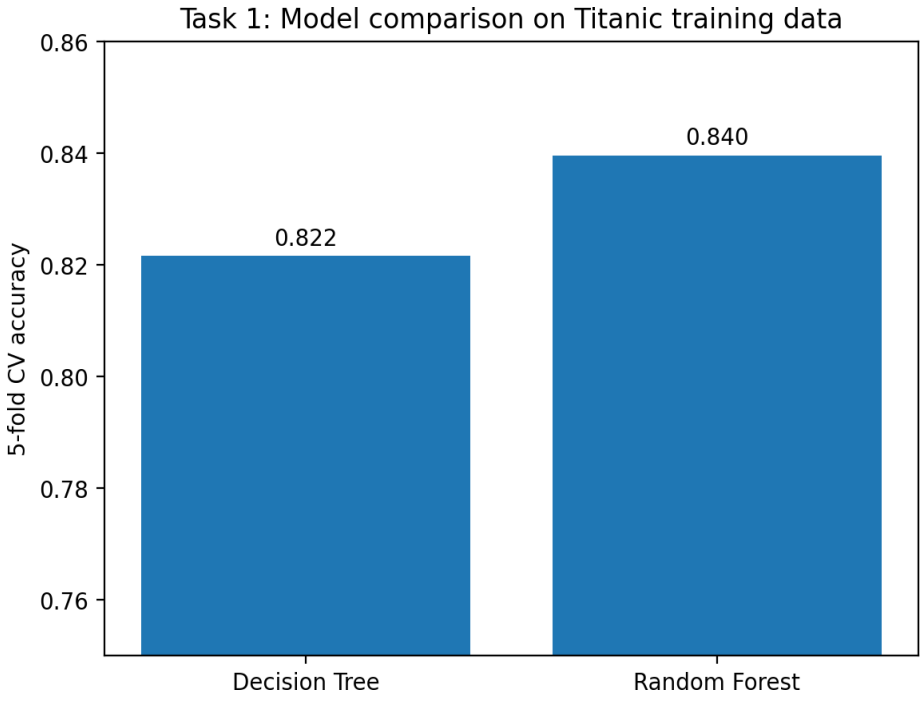

Figure 1. Five-fold cross-validation mean accuracy for the tuned Titanic models.

Task 1 figure — Fine-tuned decision tree

The plot below shows the fine-tuned decision tree fit on the full Titanic training data after preprocessing. The root split on the encoded “Mr” title feature shows that sex/title information is highly informative for survival prediction.

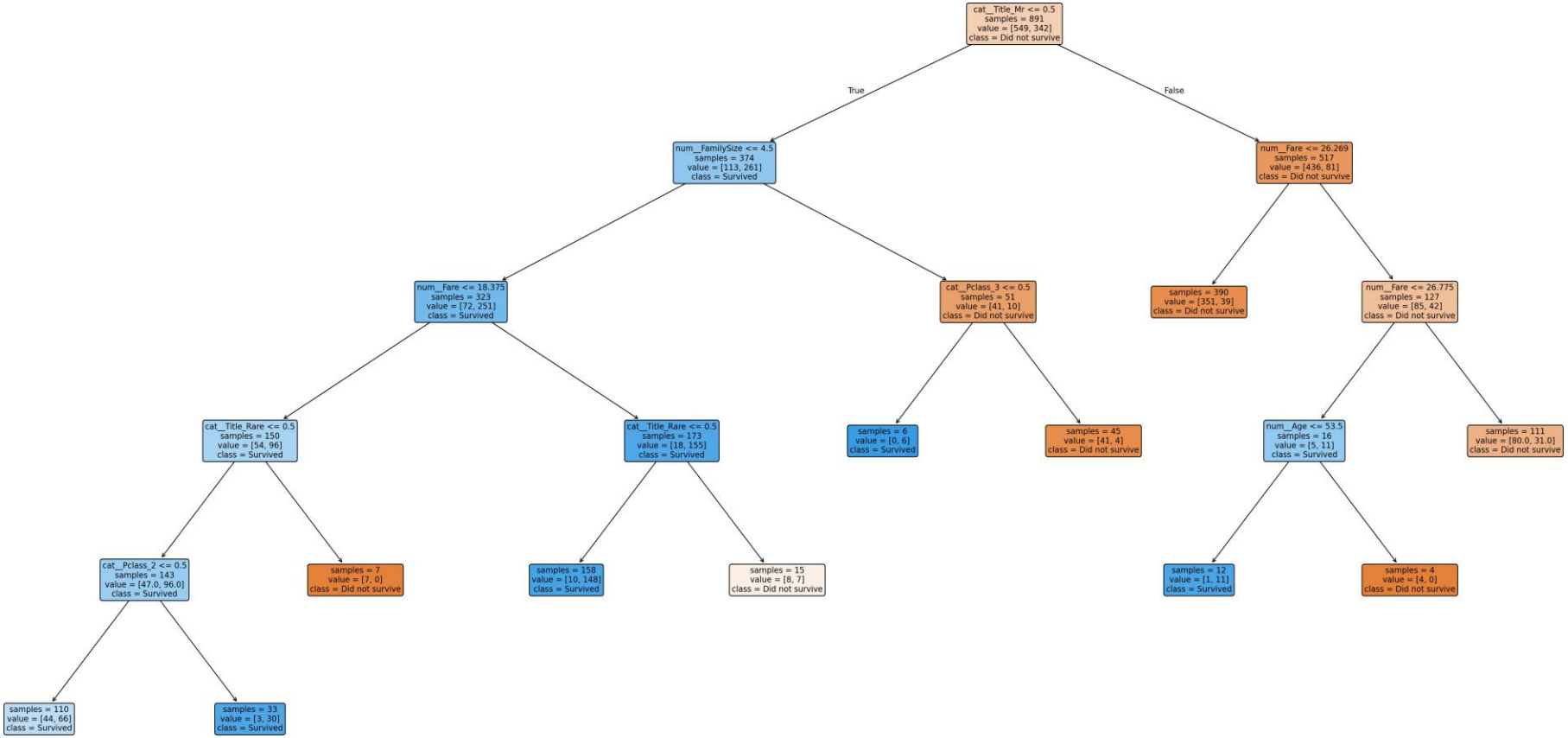

Figure 2. Decision tree learned from the preprocessed Titanic training set.

Task 2 — Training error and prediction from the given tree

(a) Training error rate. Each leaf predicts the majority class in that leaf, so the number of training errors at a leaf is the minority count.

Leaf	Counts	Predicted class	Errors
A=0, B=0, D=0	+14 / -5	+	5
A=0, B=0, D=1	+6 / -7	-	6
A=0, B=1, E=0	+2 / -10	-	2
A=0, B=1, E=1	+8 / -6	+	6
A=1, C=0	+5 / -17	-	5
A=1, C=1	+15 / -5	+	5
Total	100 records		29

Therefore the training error rate is **29 / 100 = 0.29 = 29%**.

(b) Classification of T = {A=0, B=1, C=1, D=1, E=0}. Follow the path A=0 → B=1 → E=0. The reached leaf has counts +2 and -10, so the tree assigns class **negative (-)**. Notice that C and D are not used once the path goes through E.

Task 3 — Gini before and after splitting

The dataset has 10 records: 4 positive and 6 negative.

Overall gini before splitting:

```
Gini(D) = 1 - (4/10)^2 - (6/10)^2 = 1 - 0.16 - 0.36 = 0.48.
```

Split on A:

```
A = T: 7 records → 4 positive, 3 negative → Gini = 1 - (4/7)^2 - (3/7)^2 = 24/49 ≈ 0.4898.

A = F: 3 records → 0 positive, 3 negative → Gini = 0.

Weighted gini after split on A = (7/10)(0.4898) + (3/10)(0) = 0.3429.

Gain from split on A = 0.48 - 0.3429 = 0.1371.
```

Split on B:

```
B = F: 6 records → 1 positive, 5 negative → Gini = 1 - (1/6)^2 - (5/6)^2 = 10/36 ≈ 0.2778.

B = T: 4 records → 3 positive, 1 negative → Gini = 1 - (3/4)^2 - (1/4)^2 = 6/16 = 0.375.

Weighted gini after split on B = (6/10)(0.2778) + (4/10)(0.375) = 0.3167.

Gain from split on B = 0.48 - 0.3167 = 0.1633.
```

Attribute	Weighted gini after split	Gini gain
A	0.3429	0.1371
B	0.3167	0.1633

Since splitting on **B** gives the larger gain, the decision tree would choose **B**.

Task 4 — Conceptual questions about trees

Q1. Standard decision trees are **not linear classifiers**. A linear classifier uses one global linear boundary such as $w \cdot x + b = 0$. A decision tree instead performs a sequence of local splits and partitions the feature space into rectangular (or piecewise) regions. Because the final decision boundary is made of many split segments, it is generally nonlinear.

Q2. No — misclassification error is usually not better than Gini index for choosing splits. Misclassification error is less sensitive to changes in class proportions. Gini responds more strongly when a split makes child nodes purer, so it gives more informative guidance during tree growth. For that reason, Gini (or entropy) is commonly used for splitting, while misclassification error is more often used later for pruning or overall evaluation.

Task 5 — Bagging vs. random forests

Weaknesses of bagging. Bagging reduces variance, but if many trees are still very similar, the gain from averaging is limited. This happens when a few strong predictors dominate the split decisions, causing the bagged trees to become highly correlated. Bagging also does not directly improve interpretability, and it does not reduce bias as much as variance.

Difference between bagging and random forests. Both methods use bootstrap samples, but random forests add one more source of randomness: at each split they examine only a random subset of features instead of all features.

Why this helps. The random feature subset makes the trees less correlated. Once the trees are more diverse, averaging them becomes more effective, which usually lowers variance and improves test performance compared with plain bagging.

Task 6 — SVM in the mapped space (x1, x1x2)

Use the mapping $\varphi(x1, x2) = (z1, z2) = (x1, x1x2)$. The four labeled points become:

Original point	Label	Mapped point
(-1, -1)	negative	(-1, 1)
(-1, +1)	positive	(-1, -1)
(+1, -1)	positive	(1, -1)
(+1, +1)	negative	(1, 1)

In the mapped space, the two positive points lie on the line $z2 = -1$ and the two negative points lie on the line $z2 = +1$. Therefore the maximal margin separator is the horizontal line **$z2 = 0$**, i.e. **$x1x2 = 0$**. One valid separating hyperplane is $w = (0, -1)$, $b = 0$, which gives the classifier $\text{sign}(-x1x2)$. All four points are support vectors. The distance from the separator $z2 = 0$ to either support line $z2 = \pm 1$ is **1**, so the margin is **1**.

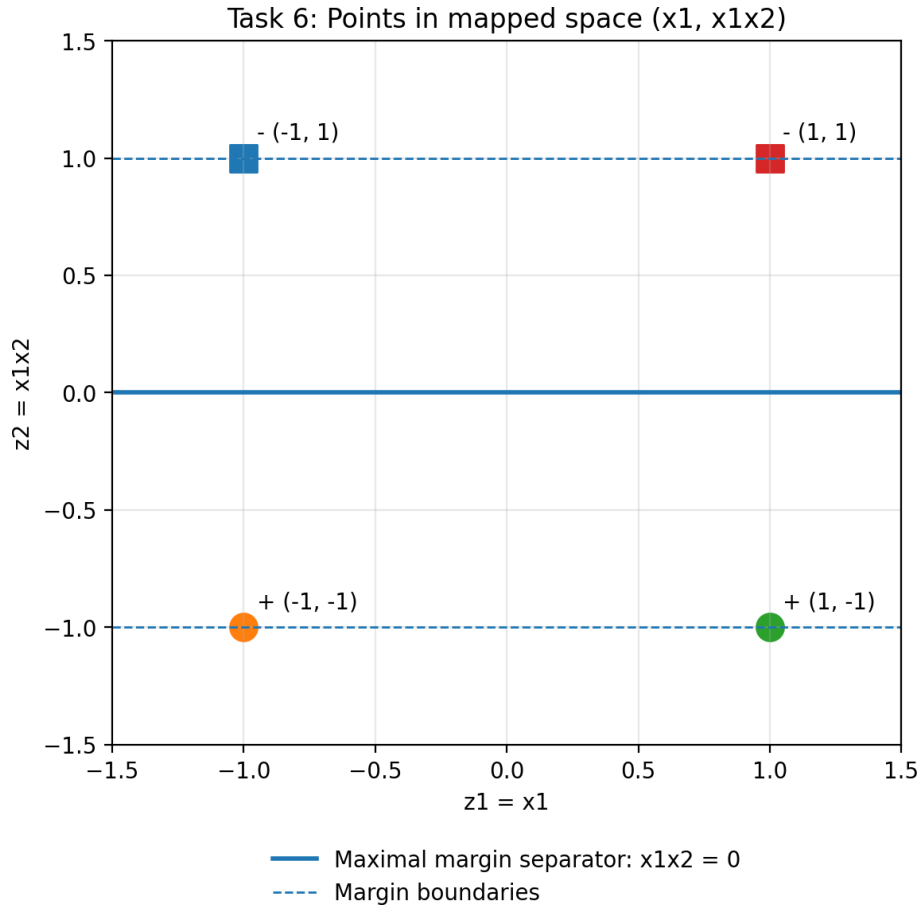

Figure 3. The mapped points, margin boundaries, and maximal margin separator.

Task 7 — Circular regions become linear in a lifted feature space

Start with the circle equation:

```
(x1 - a)^2 + (x2 - b)^2 - r^2 = 0
```

Expand it:

```
x1^2 - 2ax1 + a^2 + x2^2 - 2bx2 + b^2 - r^2 = 0
```

Rearrange by features (x1, x2, x1^2, x2^2):

```
(-2a)x1 + (-2b)x2 + (1)x1^2 + (1)x2^2 + (a^2 + b^2 - r^2) = 0
```

If $\varphi(x)$ = (x1, x2, x1^2, x2^2), then the circle boundary is a linear hyperplane **$w \cdot \varphi(x) + b0 = 0$** with w = (-2a, -2b, 1, 1) and b0 = a^2 + b^2 - r^2. Points inside the circle satisfy $w \cdot \varphi(x) + b0 \leq 0$ and points outside satisfy $w \cdot \varphi(x) + b0 > 0$. So every circular region is linearly separable from the rest of the plane in this feature space.

Task 8 — Degree-2 polynomial kernel and elliptic regions

The kernel is K(u, v) = (1 + u·v)^2. Expanding for u = (u1, u2) and v = (v1, v2):

```
K(u, v) = 1 + 2u1v1 + 2u2v2 + u1^2v1^2 + u2^2v2^2 + 2u1u2v1v2
```

This is exactly an inner product in a degree-2 feature space. An exact feature map is:

```
Φ(x) = (1, √2 x1, √2 x2, x1^2, x2^2, √2 x1x2)
```

This spans the same monomials as (1, x1, x2, x1^2, x2^2, x1x2), so a degree-2 polynomial-kernel SVM is equivalent to a linear SVM in that lifted space (up to coordinate scaling).

Now expand the ellipse equation c(x1 - a)^2 + d(x2 - b)^2 - 1 = 0:

```
c x1^2 - 2ac x1 + c a^2 + d x2^2 - 2bd x2 + d b^2 - 1 = 0
```

or equivalently

```
(ca^2 + db^2 - 1) + (-2ac)x1 + (-2bd)x2 + c x1^2 + d x2^2 + 0·x1x2 = 0
```

That is linear in the lifted feature space, so the ellipse boundary becomes a hyperplane there. Therefore SVMs with the degree-2 polynomial kernel can separate elliptic regions from the rest of the plane. The presence of the x1x2 feature also allows the same framework to represent rotated ellipses, which include a cross term.

Code URL placeholder for submission: Replace the line below with your own public GitHub / Dropbox / OneDrive / Google Drive link before turning in the homework.

https://github.com/nikhilsrikant/Data-Mining-HW2.git

*Note: a local copy of the supporting code is provided separately as **hw2_solution_code.py**. Upload that file to your preferred service and then replace the placeholder URL above.*